

Design Your Software Factory Architecture

Everyone arrives with a repository they are allowed to change. They leave with a written policy for it that a run can read, and that they could defend in a design review.

0:00 to 0:20

Their repository, not mine

Goal Each person has named the area of their codebase where an error would be most expensive.

Say

- Set the rule for the whole cohort in the first minute: every exercise happens on their code, and the lab exists only to show a worked example.
- Ask them to write down the single directory they would least like a machine to edit, and why it earns that.
- Collect three answers aloud. The reasons differ, and hearing that is what stops the policy becoming a template they copy.

Run

```
$ git clone <their repository> && cd <repository>
$ ls
```

Room does

- Opens their own project beside the lab.
- Writes one directory name and one sentence of justification in the shared document.

Ask *Would a small edit there be safer than a large one somewhere else? Say why out loud.*

0:20 to 0:45

Grading freedom by consequence

Goal They can defend a tier for their repository using recoverability rather than size.

- Say**
- Offer the two candidate axes and let the cohort argue for a few minutes before you settle it.
 - Use the comparison that ends the argument: a huge rewrite on a project nobody uses against a tiny edit to a payment path.
 - Then complicate it usefully. One tier for a whole repository is too blunt once areas differ, which is why the next exercise is per area.
-

Run

```
$ cat $REPO/.factory/charter.md
```

The rules live in the repository they govern. The code that reads them is `steps/01-boundary/policy.ts`, and it names no project.

- Room does**
- Drafts a tier line for their own repository.
 - Argues one case with a neighbour, which surfaces the disagreements early.
-

Ask *What is the worst thing an unattended run could do to you, and how long would undoing it take?*

0:45 to 1:20

Who owns which paths

Goal Everyone has a graph where every directory has exactly one owner, and the build refuses when one does not.

Say

- Introduce the graph as the thing that makes policy specific rather than vague. Areas, the paths they own, and the commands that check them.
- Build the lab's version live, then break it by adding an unclaimed directory so the room sees the refusal.
- Insist on the property that matters. Guessing an owner is worse than failing, because a wrong guess is silent.

Run

```
$ node steps/01-boundary/demo.ts graph
$ mkdir -p target/services/orphan && touch target/services/orphan/me
$ node steps/01-boundary/demo.ts graph
```

Room does

- Sees every area listed with its paths and its checks.
- Sees the second run refuse and name the directory nobody claimed.
- Writes the first version of their own graph.

Ask *Which directory in your project was hardest to assign, and what does that tell you?*

1:20 to 1:45

The number that stops production

Goal Each person has committed to a limit on pending decisions, chosen from their own week.

- Say**
- Ask how many changes they genuinely absorb in a week, then ask them to halve it for anything they did not write.
 - Explain what the limit protects: not the machine, the person. When it is reached, starting more work is the wrong move.
 - Show the refusal in the lab so the number is visibly load bearing rather than advisory.
-

Run

```
$ node steps/01-boundary/demo.ts capacity
```

- Room does**
- Watches a run decline to start because the review queue is full.
 - Puts a number in their own charter and says it aloud.
-

Ask *What happens today when that number is exceeded, and who notices first?*

1:45 to 2:00

Before Wednesday

Goal Everyone has a committed policy file in their own repository before the next session.

- Say**
- Set the deliverable precisely: committed, not drafted, because the next session reads it.
 - Warn about the common mistake, which is writing something so cautious that nothing can ever run.
-

Run

```
$ node steps/01-boundary/demo.ts lint-charter <path to theirs>
```

Room does Runs the check against their own draft and fixes what it names.

Ask *Bring the line you are least sure about to Friday.*

1:35 to 1:50

Watching a limit hold

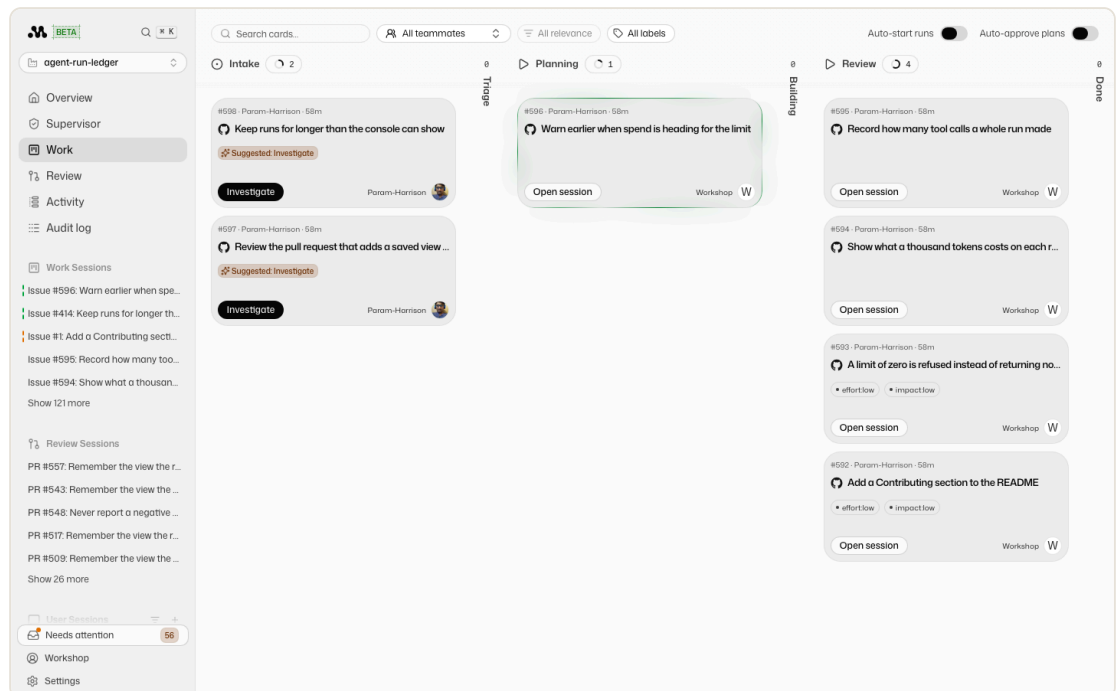
Goal Each person has seen their own class of protected path refused rather than described.

Say

- A limit that has never stopped anything is a sentence, not a limit.
- Walk the evidence backwards: no branch on the remote, no pull request, and a written reason that names the path.
- Then the uncomfortable part. Ask whether the refusal came from the charter or from the wording of the issue, because only one of those generalises.

Run

```
$ make profile NAME=enterprise
$ make lab-reset
```



The item reaches planning and stops. No branch, no pull request, no edit.

Show on the board

Work to the money item to Planning to its session

Read the reason, not just the refusal

A run that stops because the issue said "a person decides this one" looks identical to one that stops because the charter protects the path. Only the second generalises to their repository, and the difference is a sentence in the record.

Nothing was written

Check the remote for a branch, then for a pull request. A refusal that leaves either behind is not a refusal, and this is the check to run on their own factory.

Protected and refuse are different stops

A refused target stops after planning with a plan nobody may act on. A protected path stops before planning starts. Enterprise uses both and scaleup uses one.

Room does

- Applies a shape to their own repository and names one path it would refuse.
- Checks the remote afterwards and confirms nothing was created.

Ask

Which of your protected paths has ever actually stopped a change? If none, is it protecting anything?

Concepts: the charter, autonomy by consequence, protected paths, back-pressure, the ownership graph.